

Extended Abstract

Motivation While large language models (LLMs) perform well at code generation, they struggle with bug fixes due to the lack of bug-fix examples in pretraining datasets. Furthermore, code generation benchmarks are becoming saturated, limiting their utility for evaluating models. We consider the problem setting in which we have a test set of very hard bugs that we want an LLM to fix. The model must bootstrap its capabilities by generating helpful bugs that it can then learn to fix. Our work is inspired by test-time training and meta-learning techniques, where knowledge of the test distribution is available during training.

Method We study whether targeted curriculum generation can improve the model’s bug-fixing capabilities on the target bugs. Specifically, we fine-tune a code model using Direct Preference Optimization (DPO) on synthetic bugs, using several methods for weighting training examples based on their usefulness to the fixed target set. These methods include: target loss improvement after one gradient step, gradient alignment between train and test losses, GPT-4o-provided helpfulness scores, reward-weighted regression using these scores, and model entropy as a proxy for training bug difficulty.

Implementation We fine-tune Qwen2.5-7B-Coder-Instruct using DPO and synthetic bugs derived from the BigCodeBench dataset. We generate 939 target bugs using GPT-4o and construct a synthetic training set of 1500 bugs unsolvable by the base model. Preferences are constructed using pass@10 scores, and each auxiliary method modulates DPO loss based on the relevance of a training bug to the test set.

Results Vanilla DPO, Target Loss, and Entropy achieved the highest test bug performance, with a Pass@1 of 4.9, significantly outperforming the baseline (0.0). Interestingly, despite relying on different signals such as target-aware, target-free, and uncertainty-based, these three methods solved the same bugs, suggesting DPO’s preference signal is well-aligned already. Gradient alignment achieved the best training bug performance but showed tradeoffs in test bug generalization. GPT-4o and RWR led to more divergent solutions but did not outperform simpler approaches. Our results show that while test-aware signals can diversify behavior, they often introduce training noise.

Discussion Our results reveal that simple auxiliary losses for targeted bug generation may not always outperform simple preference-based fine-tuning, particularly when preference signals already align with task objectives. Methods such as gradient alignment and reward-weighted regression introduce useful diversity but can destabilize training due to noisy gradient estimates or unaligned helpfulness scores. Time and compute limitations ultimately constrained our ability to explore broader solver integration and longer training. Nonetheless, our findings suggest that scalable, label-free fine-tuning using synthetic data remains a promising direction.

Conclusion We show that test-time training on synthetic bugs can improve model performance on fixed, difficult test bugs, but that simple DPO may suffice when guided by a good preference signal. Our results indicate that incorporating noisy targeted generation heuristics can harm rather than help, unless carefully tuned. Future work could explore combining complementary methods through ensembling or curriculum scheduling and applying these techniques in agentic, interactive debugging pipelines.

Test-Aware Training for Targeted Bug Fixing

Evelyn Choi

Department of Computer Science
Stanford University
echoi1@stanford.edu

Abstract

Although LLMs have shown strong code generation skills, they struggle with bug fixing, especially on novel bugs, due to the under representation of bug-fix examples in pretraining datasets. To tackle this, we implement test-aware training, where we use the knowledge of a fixed target set of bugs during training to improve the model’s test performance. We specifically finetune Qwen2.5-7B using Direct Preference Optimization on synthetically generated bugs with BigCodeBench. We experiment with different target-aware weighting schemes on top of our DPO loss, such as target loss, gradient alignment, entropy, and LLM-generated helpfulness scores. We found that vanilla DPO improves performance on the target bug set, and that Pass@1 increased by 4.9% from 0% on the test set of difficult bugs. Our results reveal that simple auxiliary losses for targeted bug generation may not always outperform simple preference-based fine-tuning, particularly when preference signals already align with task objectives. However, this work demonstrates that LLMs can improve in a scalable manner with only synthetic data without human supervision or labels.

1 Introduction

Language models have recently demonstrated impressive performance on code generation and new, complex coding benchmarks (Chen et al., 2021) (Zheng et al., 2024) (Xia and Zhang, 2022). However, although LMs are skilled at code generation, they tend to struggle with debugging and code understanding. This is often due to examples of bug fixes being underrepresented in pretraining datasets, making it difficult for LMs to generalize to novel bugs. However, manually collecting or scraping real-world bug-fixes can be expensive and time-consuming.

Furthermore, many existing code generation benchmarks are becoming saturated by modern LLMs, curbing their usefulness for future evaluation. One promising direction involves using synthetic data to train models on unseen or difficult bugs. However, this is a difficult task. We can see in Figure 1 that LLMs often fail to distinguish between solvable and unsolvable bugs. Therefore, we need training signals from actual results or unittests.

In this paper, we explore the following context: suppose we have access to an unlabeled set of difficult target bugs that we ultimately want to solve. Can we train an LLM to solve these bugs by using this knowledge during training? Our objective in this work is to explore whether test-aware training can improve an LM’s performance on a set of target bugs. Specifically, we examine how to most effectively learn from or weigh training bugs when our test set is known during training.

Recent work has explored synthetic data generation and RL finetuning for LLM code generation (SWE-Bench, SWE-Smith). However, these works do not target an explicit set of fixed bugs during training. Prior work on test-time training or meta-learning, such as gradient alignment, have not been applied to code debugging.

Our aim in this paper is to explore the following research questions. Can we improve an LLM’s ability to fix specific target bugs using synthetic data? What target-aware methods or reward signals are most effective? Are these additional target-aware signals necessary, or is simple preference-based RL finetuning sufficient?

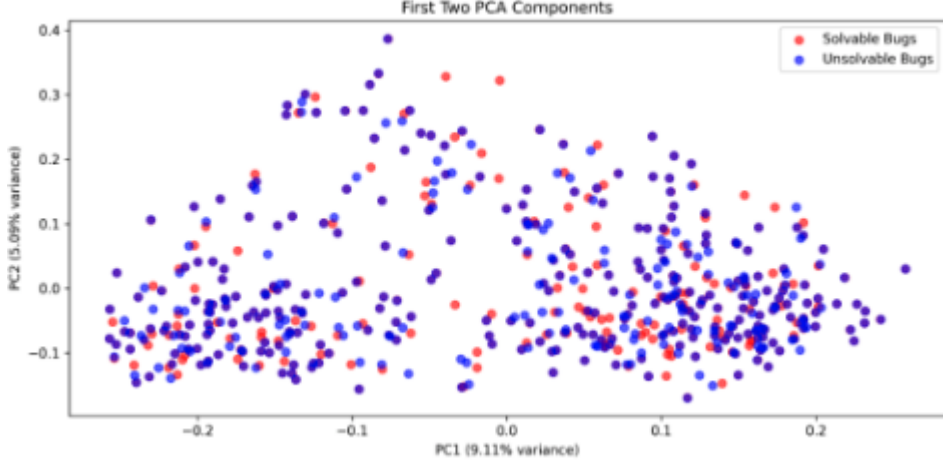


Figure 1: Visualization of LLM embeddings of solvable vs unsolvable bugs.

2 Related Work

Synthetic Data for Coding To address data scarcity and enable scalable training, existing methods utilize powerful language models (LMs) to generate extensive synthetic datasets for code-related tasks Nijkamp et al. (2023); Li et al. (2023). Earlier works, such as CodeAlpaca and CodeLlama, create instruction-tuning datasets by distilling outputs from sophisticated teacher models like GPT-4, starting from limited task examples University (2023); Roziere et al. (2023). To increase the diversity and difficulty of these synthetic datasets, previous approaches employ manually-crafted heuristics Luo et al. (2023) or prompt LLMs using open-source code snippets Wei et al. (2023). At the repository level, SWE-Smith and SWE-Synth Yang et al. (2025); Pham et al. (2025) generate bug-fix tasks with the help of advanced teacher models, such as Claude-3.7. However, these methods rely on static, manually-engineered generation processes. In contrast, our method uses adversarial self-play to dynamically learn an adaptive bug-generation policy, eliminating the reliance on external teacher models.

Test-Time Training and Adaptation Test-time training (TTT) or adaptation approaches adjust model parameters or outputs during inference, typically by leveraging information from the test distribution itself. Such methods have shown promise in improving generalization to novel or shifted data distributions. For instance, (Sun et al., 2020) introduced a TTT framework where models self-supervise on auxiliary tasks during testing to adapt dynamically to new distributions. Wang et al. (2020) proposed Tent, which updates batch normalization parameters at test-time to mitigate distribution shifts. Despite their success in vision and NLP tasks, these techniques have seen limited exploration in code-related applications, particularly debugging. Gradient alignment (Nichol et al., 2018), a meta-learning approach closely related to test-time training, aligns training gradients with gradients computed from target tasks to enhance model generalization. Although gradient alignment has been effective in few-shot and domain adaptation scenarios, it has not yet been applied to explicitly improve debugging performance on code tasks. Our work differs by explicitly targeting a predefined set of difficult code bugs during training and employs synthetic data generation strategies guided by test-aware signals. This targeted approach allows us to directly assess the benefits of test-time signals and compare them against traditional reinforcement learning finetuning methods for debugging tasks.

3 Method

To finetune our model, we use Direct Preference Optimization (DPO) on a synthetic training set with different auxiliary rewards (Rafailov et al., 2024). The exact method for generating this synthetic training set is outlined in the next section. To generate preferences from our synthetic bugs, we score each bug with the pass@10 from running our base model on it. We then construct preference pairs, giving preference to lower scored bugs, i.e. bugs that are more difficult for our base model to solve, with a preference threshold of 0.4.

Let $\mathcal{B}, \mathcal{T}$ denote our training and target bugs, respectively. In this paper, we use target and test bugs synonymously. Then consider a batch of synthetic bugs $B \in \mathcal{B}$ and target bug $T \in \mathcal{T}$. Our total loss can be described as:

$$L_{\text{total}}(B, T; \theta) = \frac{1}{N} \sum_{i=1}^N (1 + \alpha H(B_i, T)) \ell_{\text{DPO}}(B_i; \theta) \quad (1)$$

where H is some function mapping pairs of training and target bugs to a score describing how helpful that training bug is to solving our target bugs. We explore the following methods for determining test helpfulness H .

3.1 Target Loss

Our first method to encode helpfulness is target loss. That is, we define helpfulness based on how much solving a training bug improves the model’s loss on the target bug after a single gradient step. Here we denote θ_0 to be the original parameters of the model, and θ_{0+B_i} denotes taking one gradient step on the train bug. We define:

$$H_{\text{target loss}}(B_i, T) = \ell_T(\theta_0) - \ell_T(\theta_{0+B_i}) \quad (2)$$

where the loss ℓ denotes a negative log likelihood loss.

3.2 GPT-4o

We also experiment with an LLM-based proxy for helpfulness by prompting GPT-4o. We prompt the model with both the training and test bugs and ask it to rate how solving the training bug might help with solving the test bug:

$$H_{\text{GPT-4o}}(B_i, T) = \text{GPT4o}(B_i, T) \quad (3)$$

We wanted to see how useful an LLM’s assessment would be for the model.

3.3 Reward-Weighted Regression (RWR)

We also experiment with Reward-weighted Regression (RWR) (Peters and Schaal, 2007). Instead of equation 1, in this case we instead use the total loss defined as follows:

$$L_{\text{total}}(B, T; \theta) = \frac{1}{N} \sum_{i=1}^N (\ell_{\text{DPO}}(B_i; \theta) + \alpha \ell_{\text{RWR}}(B_i, T; \theta)) \quad (4)$$

$$\ell_{\text{RWR}}(B_i, T; \theta) = e^{\gamma r_i(B_i, T)} \ell_{B_i}(\theta) \quad (5)$$

Here $r_i(B_i, T)$ is simply our GPT-4o score from equation 3, and γ and α are some hyperparameters. $\ell_{B_i}(\theta)$ denotes the Negative log-likelihood (NLL) on bug B_i . Since DPO provides the model with a contrastive reward signal, adding RWR also provides our model with an absolute signal. Thus this differs from the previous methods in that we are not upweighting DPO preferences for high-reward bugs, but also attempting to maximize the NLL of high-reward samples.

3.4 Gradient Alignment

Another existing method is gradient alignment, which measures the alignment between the gradients of the train and test bugs with respect to the initial model parameters θ_0 .

$$H_{\text{Grad Alignment}}(B_i, T) = \text{Cosine Similarity}(\nabla_{\theta} \ell_{B_i}(\theta_0), \nabla_{\theta} \ell_T(\theta_0)) \quad (6)$$

We essentially aim to reduce the target bug loss, but this method is more computationally efficient than the target loss method outlined in equation 2 since it doesn’t require a full update step.

3.5 Entropy

For our last helpfulness score method, we use entropy. We define it briefly as follows:

$$H_{\text{Entropy}}(B_i) = - \sum_{x=1}^V p_i(x) \log p_i(x) \quad (7)$$

where $p_i(x)$ is the distribution created by taking the softmax of the model’s final output logit for each token x in its vocabulary when fed B_i . We then normalize these entropies across the batch and zeroing out values below the batch mean so that we only reward uncertainty that is above average. We also add clipping for training stability. Here we are measuring the models’ uncertainty over a training bug as a proxy for its difficulty. Note that we do not involve the target bug here. Therefore, this allows us to evaluate the benefit of having access to unlabeled target bugs before test time.

4 Experimental Setup

For our base model, we use the Qwen2.5-7B-Coder-Instruct (Hui et al., 2024). In our setup, we sourced our synthetic data from the BigCodeBench benchmark (Zhuo et al., 2025). This benchmark challenges LLMs to call on multiple function calls as tools from 139 libraries and 7 domains for around 1,000 tasks.

We used GPT-4o (OpenAI et al., 2024) to generate around 939 target bugs for testing. This test set is constructed from 415 BigCodeBench code generation problems. We prompt GPT-4o to modify the ground-truth code from BigCodeBench so that it causes some unit tests to fail. We then check that the bug is “valid” – that is, the buggy code compiles and at least one unit test fails. Only valid bugs are included in our train and test sets. We filtered to make sure that these target bugs were unsolvable in 5 attempts by the base Qwen model. For our training bugs, we generated 1500 synthetic bugs using Qwen on the remaining 725 BigCodeBench problems. We compare against the base Qwen2.5-7B-Coder-Instruct model and one iteration of generator-solver training. We evaluate pass @ 1 on the test bug fix dataset for all baselines.

We use the TRL library by von Werra et al. (2020) and the vLLM library by Kwon et al. (2023) for bug generation and training.

5 Results

5.1 Quantitative Evaluation

Table 1 shows our results for each of the methods outlined in Section 3. Note that due to the nature of our synthetic dataset construction, we expect our base Qwen model to be unable to solve any test bugs. We observe that vanilla DPO finetuning, as well as DPO with target loss and entropy did the best on the test bugs, while entropy and gradient alignment did the best on the train bugs. In addition, all methods beat the baseline on both the test and train bugs. This shows that our synthetic data was useful and that finetuning increases the model’s ability to fix both train and target bugs. In addition, Vanilla DPO, Target Loss, and Entropy all achieved the highest test performance of 4.9, meaning they were able to solve 47 previously unsolvable bugs. This is interesting, as these methods all rely on different assumptions. Notably, Target Loss uses the test bugs during training, while vanilla DPO and Entropy do not. The strong results of Entropy could suggest that model uncertainty is a good heuristic for knowing which training bugs can support better generalization. However, as mentioned in Figure 3, these three methods all solved exactly the same test bugs. This could instead indicate that Target Loss and Entropy simply reinforce the learning signals from DPO and thus do not increase model generalization.

For all of the results in Table 1, we did a short hyperparameter search across a couple different values of α for each method and chose the model with the best performance. We could not extend our hyperparameter search for all models due to compute constraints. However, we explore the effects of

Table 1: Pass@1 of each method on train and test bugs.

Method	Test Bugs	Train Bugs
Original Model (Baseline)	0	9.94
Vanilla DPO	4.90	15.50
Gradient Alignment	4.80	15.90
GPT-4o	4.37	15.12
Target Loss	4.90	15.84
Entropy	4.90	15.91
Reward-Weighted Regression	4.37	15.62

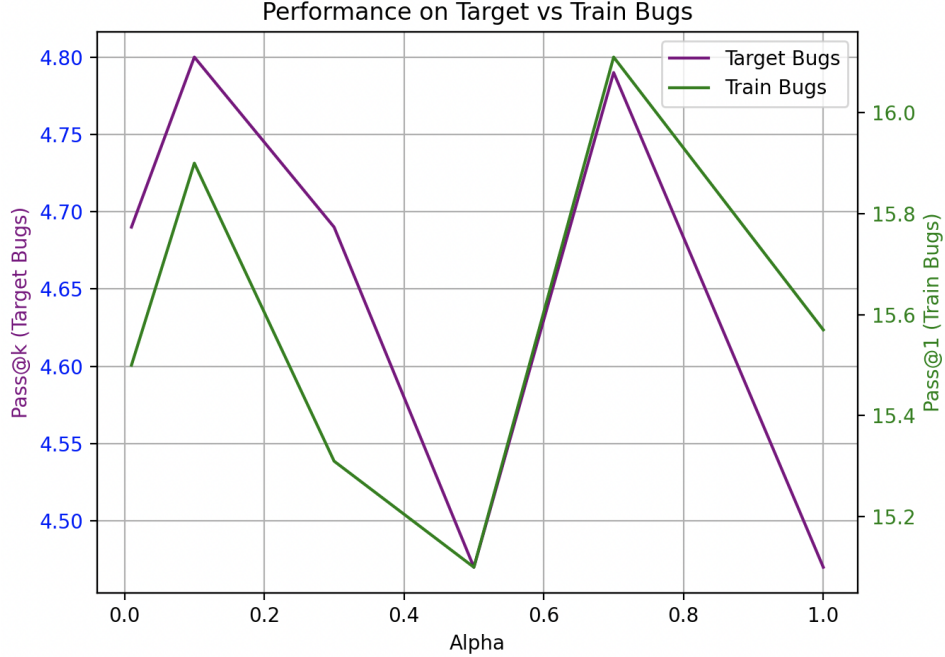


Figure 2: Test and Train Evaluation (Pass@1) of Grad Alignment over different values of α .

α in more depth in Figure 2, where we show the results of 6 different values of alpha on the model’s train and test performance for the gradient alignment method. A further analysis of this is in the discussion section.

In addition, Figure 3 shows the number of test and train bugs that either vanilla DPO or a given method was able to solve, but not the other. Not included in the graphs are the bars for Target Loss and Entropy, which matched vanilla DPO on the test bug performance. This is because these two methods were able to solve exactly the same test bugs as vanilla DPO, and there was no difference in test bugs solved between the three.

5.2 Qualitative Analysis

Figure 4 shows the difference in bug types that our base model generates before and after vanilla DPO finetuning. For this figure, we selected 100 snippets of correct code from BigCodeBench (ground truth solutions) and asked the Qwen model to insert a bug for each snippet. We then used GPT-4o to classify each bug types with special prompting.

A code snippet of an example test bug is shown in Figure 5. Before vanilla DPO, the model was unable to fix the bug outlined in red. After finetuning, the model fixed the code and passed all unit tests.

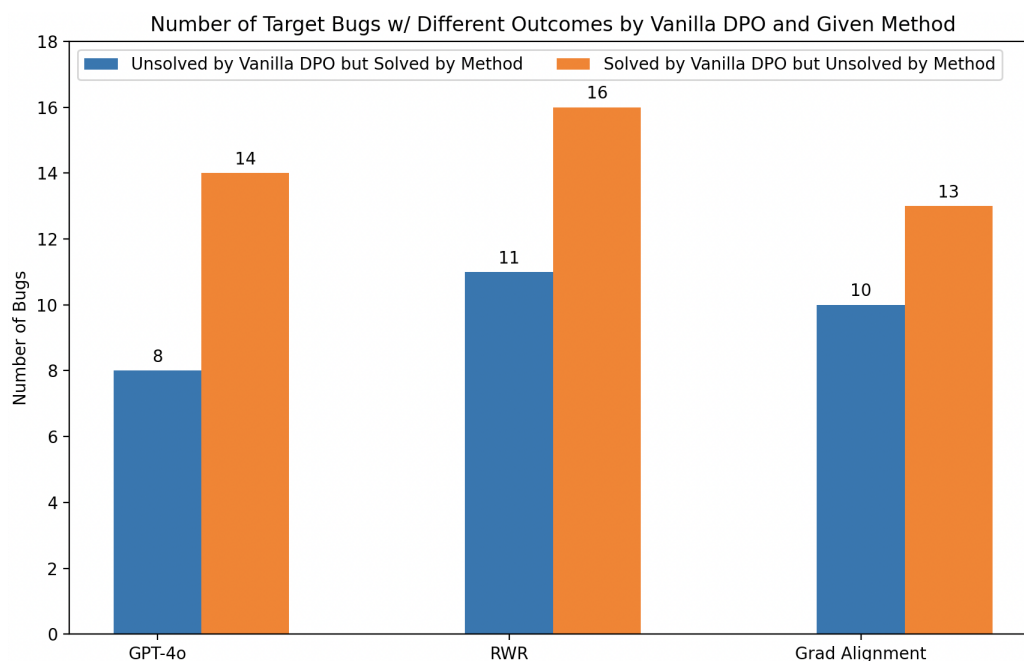


Figure 3: Visualization of Target Bugs with Different Outcomes (missing methods solved the same bugs as vanilla DPO).

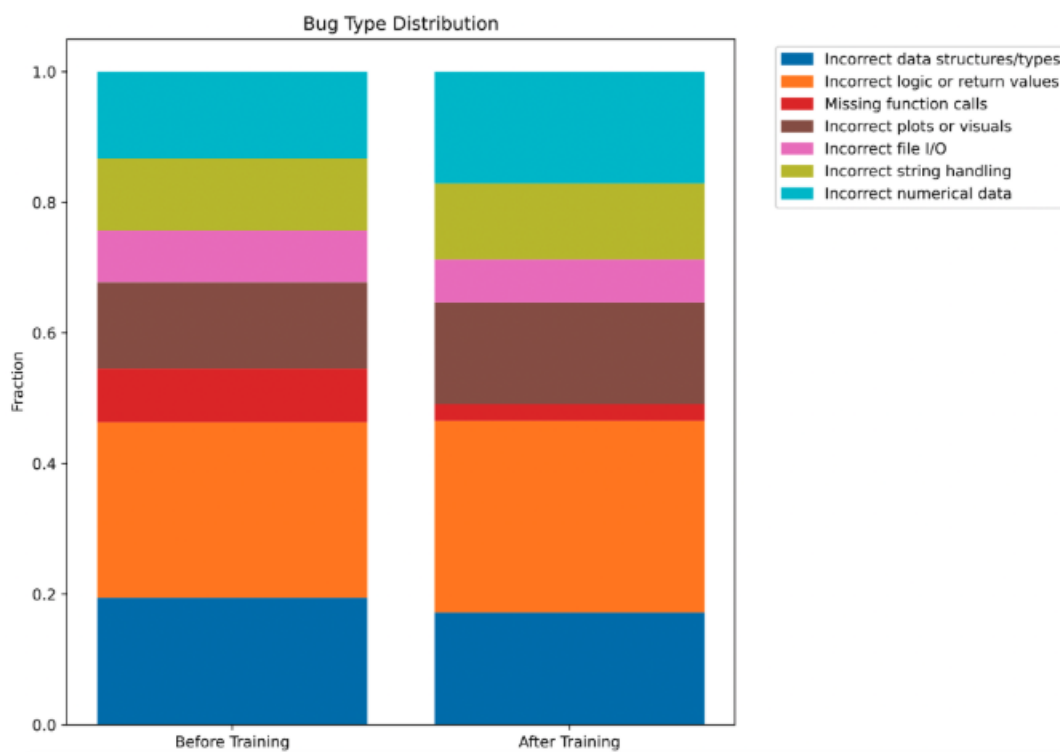


Figure 4: Visualization of generated bug type distribution before and after vanilla DPO.

```

from datetime import datetime
import matplotlib.pyplot as plt
import pandas as pd
def task_func(data):
    # Handle empty data
    if not data.strip():
        raise ValueError("The provided data string is empty.")

    data_entries = data.split(',')
    months_data = [d.split('-')[1] for d in data_entries]
    unique_years = {d.split('-')[0] for d in data_entries}

    # Check if the data is from the same year
    if len(unique_years) != 1:
        raise ValueError("The provided data contains entries from multiple years.")

    # Extract data and convert to DataFrame
    data = [d.rsplit('-', 1) for d in data_entries]
    data = [(datetime.strptime(d[0], '%Y-%m').strftime('%B'), int(d[1])) for d in data]
    df = pd.DataFrame(data, columns=['Month', 'Value'])
    df = df.set_index('Month')

    fig, ax = plt.subplots(figsize=(10, 6))
    ax.bar(df.index, df['Value'])
    ax.set_xlabel('Month')
    ax.set_ylabel('Value')
    ax.set_title(f'Monthly Data for {list(unique_years)[0]}')
    plt.xticks(rotation='vertical')
    plt.close(fig) # Close the figure to prevent it from being displayed here

    return ax

```

Figure 5: Example Target Bug Vanilla DPO Learned to Solve

Figure 3 also shows that GPT-4o, RWR, and Gradient Alignment lead to different model behavior, unlike Target Loss and Entropy. This is because each of these methods solves bugs that vanilla DPO was unable to solve, and also fails to solve bugs that vanilla DPO is able to solve, and thus these methods seem to be guiding the model to optimize differently than vanilla DPO. Gradient alignment showed the largest net gain, with 10 bugs solved that vanilla DPO could not solve, and 13 bugs that it failed on. Thus it seems that it is less noisy than GPT-4o and RWR. However, all of these methods ultimately have a net loss, showing that although these helpfulness scores guide the model to optimize differently lead to more diverse behavior, this causes them to forget useful behaviors from DPO.

The fact that no methods outperformed vanilla DPO actually suggests that our DPO preference signal is already well aligned to our task of increasing target bug performance. Our DPO preference signal, i.e. the model’s pass@10 scores, makes the model assign higher likelihood to bugs that are more difficult for the model. Therefore, it’s possible that this is sufficient for improving target bug performance. In addition, our helpfulness scores are often noisy could negatively impact or destabilize training. We observed this noisiness for gradient alignment in our loss during training.

However, vanilla DPO gets the third-worst performance on the train bugs, which could just mean it overfits the train bugs less than the methods augmented with a helpfulness score.

In addition, our GPT-4o scores may not be well aligned with our model, which could conflict or add noise to our DPO loss. That is, what GPT-4o might think is helpful for our target bugs may not be true for our actual model. Hence it makes sense that both GPT-4o and RWR had the worst test performance. However, there wasn’t much difference in test performance between the two, even though RWR added an absolute reward while GPT-4o simply weighted the DPO loss. However, RWR did fit slightly more to the train bugs than GPT-4o.

The purpose of 4 was to see whether DPO finetuning could change the model’s output bug distribution across different bug types. We can see that there aren’t any changes in the frequencies of any bug types, and that all bug types seem to be proportional to what they were before training. This has useful implications for curriculum learning, as we are seeing performance increases without a decrease in output bug diversity.

6 Discussion

The limitations and difficulties during this project were mainly due to time and compute constraints. We mainly trained our models on generating better bugs, and didn’t also generate synthetic solution data to train a solver end-to-end. This was mainly due to compute constraints, since training each

model took a while. Therefore, one natural extension of our work would be to take our current finetuned generator model, generate bugs with it, and then generate solutions and train a solver with these solutions.

Another limitation is our limited hyperparameter search. We only conducted a limited search over different values of α for each method due to compute constraints. One source of the training instability in our methods is the sensitivity to the choice of α . In Figure 2 we see that the choice of α can largely impact the model’s performance on test and train bugs. This is most likely due to grad alignment producing noisy gradient estimates for some bugs, making it sensitive to the choice of α . Hence, as seen in Figure 2, this probably led to underperformance in some of our models.

Overall, our work demonstrates that simple preference-based finetuning (i.e. DPO) is a powerful and stable strategy for improving performance on difficult bugs. In addition, we trained only on synthetic data, suggesting that LLMs can improve without human supervision in a scalable manner, without human labels, or handcrafted rewards. In addition, while methods using test-aware optimization may be powerful in theory, they can be noisy and destabilize training, leading to subpar results.

7 Conclusion

Our work ultimately shows that LLMs can improve in a scalable manner with only synthetic data without human supervision or labels. Our results indicate that incorporating target-aware heuristics can harm performance unless carefully tuned.

One future direction would be to use an ensembling approach. We found that methods like gradient alignment were able to solve problems vanilla DPO was unable to. Thus combining these methods with vanilla DPO in an ensembling approach could boost performance. Another future exploration would be to also explore this direction in a more agentic setting, as right now our code is more one-shot in nature. We first attempted to do this using SWE-Agent experiments, and implemented a version of the pipeline with Qwen7b. However, due to time and compute constraints, we had to switch to BigCodeBench. We found that simply generating agent trajectories with Qwen7b was too time-consuming, and Qwen7b failed most of the problems, so it was impossible to generate synthetic code for self-improvement.

8 Team Contributions

- **Evelyn Choi:** I implemented and trained the models on DPO as well as the auxiliary rewards or helpfulness scores. I also implemented Qwen in Swe-Agent or the agentic setting and generated trajectories and bugs.
- I also worked with an external collaborator, Caroline Choi, whose ongoing synthetic code repository I worked out of. She set up most of the synthetic dataset pipelines and generated the GPT-4o bugs and scores.

Changes from Proposal I pivoted project topics after the milestone, so this entire project and report is different from my proposal and milestone.

References

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>

- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2. 5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- Raymond Li, Lewis Tunstall, et al. 2023. StarCoder: May the source be with you! Hugging Face BigCode Project.
- Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568* (2023).
- Alex Nichol, Joshua Achiam, and John Schulman. 2018. On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999* (2018).
- Erik Nijkamp et al. 2023. CodeGen2: Lessons for Training LLMs on Programming and Natural Languages. *arXiv preprint arXiv:2305.02309* (2023).
- OpenAI, :, Aaron Hurst, Adam Lerer, Adam P. Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, Aleksander Mądry, Alex Baker-Whitcomb, Alex Beutel, Alex Borzunov, Alex Carney, Alex Chow, Alex Kirillov, Alex Nichol, Alex Paino, Alex Renzin, Alex Tachard Passos, Alexander Kirillov, Alexi Christakis, Alexis Conneau, Ali Kamali, Allan Jabri, Allison Moyer, Allison Tam, Amadou Crookes, Amin Tootoochian, Amin Tootoochian, Ananya Kumar, Andrea Vallone, Andrej Karpathy, Andrew Braundstein, Andrew Cann, Andrew Codisoti, Andrew Galu, Andrew Kondrich, Andrew Tulloch, Andrey Mishchenko, Angela Baek, Angela Jiang, Antoine Pelisse, Antonia Woodford, Anuj Gosalia, Arka Dhar, Ashley Pantuliano, Avi Nayak, Avital Oliver, Barret Zoph, Behrooz Ghorbani, Ben Leimberger, Ben Rossen, Ben Sokolowsky, Ben Wang, Benjamin Zweig, Beth Hoover, Blake Samic, Bob McGrew, Bobby Spero, Bogo Gierler, Bowen Cheng, Brad Lightcap, Brandon Walkin, Brendan Quinn, Brian Guarraci, Brian Hsu, Bright Kellogg, Brydon Eastman, Camillo Lugaresi, Carroll Wainwright, Cary Bassin, Cary Hudson, Casey Chu, Chad Nelson, Chak Li, Chan Jun Shern, Channing Conger, Charlotte Barette, Chelsea Voss, Chen Ding, Cheng Lu, Chong Zhang, Chris Beaumont, Chris Hallacy, Chris Koch, Christian Gibson, Christina Kim, Christine Choi, Christine McLeavey, Christopher Hesse, Claudia Fischer, Clemens Winter, Coley Czarnecki, Colin Jarvis, Colin Wei, Constantin Koumouzelis, Dane Sherburn, Daniel Kappler, Daniel Levin, Daniel Levy, David Carr, David Farhi, David Mely, David Robinson, David Sasaki, Denny Jin, Dev Valladares, Dimitris Tsipras, Doug Li, Duc Phong Nguyen, Duncan Findlay, Edede Oiwoh, Edmund Wong, Ehsan Asdar, Elizabeth Proehl, Elizabeth Yang, Eric Antonow, Eric Kramer, Eric Peterson, Eric Sigler, Eric Wallace, Eugene Brevdo, Evan Mays, Farzad Khorasani, Felipe Petroski Such, Filippo Raso, Francis Zhang, Fred von Lohmann, Freddie Sulit, Gabriel Goh, Gene Oden, Geoff Salmon, Giulio Starace, Greg Brockman, Hadi Salman, Haiming Bao, Haitang Hu, Hannah Wong, Haoyu Wang, Heather Schmidt, Heather Whitney, Heewoo Jun, Hendrik Kirchner, Henrique Ponde de Oliveira Pinto, Hongyu Ren, Huiwen Chang, Hyung Won Chung, Ian Kivlichan, Ian O’Connell, Ian O’Connell, Ian Osband, Ian Silber, Ian Sohl, Ibrahim Okuyucu, Ikai Lan, Ilya Kostikov, Ilya Sutskever, Ingmar Kanitscheider, Ishaan Gulrajani, Jacob Coxon, Jacob Menick, Jakub Pachocki, James Aung, James Betker, James Crooks, James Lennon, Jamie Kiros, Jan Leike, Jane Park, Jason Kwon, Jason Phang, Jason Teplitz, Jason Wei, Jason Wolfe, Jay Chen, Jeff Harris, Jenia Varavva, Jessica Gan Lee, Jessica Shieh, Ji Lin, Jiahui Yu, Jiayi Weng, Jie Tang, Jieqi Yu, Joanne Jang, Joaquin Quinonero Candela, Joe Beutler, Joe Landers, Joel Parish, Johannes Heidecke, John Schulman, Jonathan Lachman, Jonathan McKay, Jonathan Uesato, Jonathan Ward, Jong Wook Kim, Joost Huizinga, Jordan Sitkin, Jos Kraaijeveld, Josh Gross, Josh Kaplan, Josh Snyder, Joshua Achiam, Joy Jiao, Joyce Lee, Juntang Zhuang, Justyn Harriman, Kai Fricke, Kai Hayashi, Karan Singhal, Katy Shi, Kavin Karthik, Kayla Wood, Kendra Rimbach, Kenny Hsu, Kenny Nguyen, Keren Gu-Lemberg, Kevin Button, Kevin Liu, Kiel Howe, Krithika Muthukumar, Kyle Luther, Lama Ahmad, Larry Kai, Lauren Itow, Lauren Workman, Leher Pathak, Leo Chen, Li Jing, Lia Guy, Liam Fedus, Liang Zhou, Lien Mamitsuka, Lilian Weng, Lindsay McCallum, Lindsey Held, Long Ouyang, Louis Feuvrier, Lu Zhang, Lukas Kondraciuk, Lukasz Kaiser, Luke Hewitt, Luke

- Metz, Lyric Doshi, Mada Aflak, Maddie Simens, Madelaine Boyd, Madeleine Thompson, Marat Dukhan, Mark Chen, Mark Gray, Mark Hudnall, Marvin Zhang, Marwan Aljube, Mateusz Litwin, Matthew Zeng, Max Johnson, Maya Shetty, Mayank Gupta, Meghan Shah, Mehmet Yatbaz, Meng Jia Yang, Mengchao Zhong, Mia Glaese, Mianna Chen, Michael Janner, Michael Lampe, Michael Petrov, Michael Wu, Michele Wang, Michelle Fradin, Michelle Pokrass, Miguel Castro, Miguel Oom Temudo de Castro, Mikhail Pavlov, Miles Brundage, Miles Wang, Minal Khan, Mira Murati, Mo Bavarian, Molly Lin, Murat Yesildal, Nacho Soto, Natalia Gimelshein, Natalie Cone, Natalie Staudacher, Natalie Summers, Natan LaFontaine, Neil Chowdhury, Nick Ryder, Nick Stathas, Nick Turley, Nik Tezak, Niko Felix, Nithanth Kudige, Nitish Keskar, Noah Deutsch, Noel Bundick, Nora Puckett, Ofir Nachum, Ola Okelola, Oleg Boiko, Oleg Murk, Oliver Jaffe, Olivia Watkins, Olivier Godement, Owen Campbell-Moore, Patrick Chao, Paul McMillan, Pavel Belov, Peng Su, Peter Bak, Peter Bakkum, Peter Deng, Peter Dolan, Peter Hoeschele, Peter Welinder, Phil Tillet, Philip Pronin, Philippe Tillet, Prafulla Dhariwal, Qiming Yuan, Rachel Dias, Rachel Lim, Rahul Arora, Rajan Troll, Randall Lin, Rapha Gontijo Lopes, Raul Puri, Reah Miyara, Reimar Leike, Renaud Gaubert, Reza Zamani, Ricky Wang, Rob Donnelly, Rob Honsby, Rocky Smith, Rohan Sahai, Rohit Ramchandani, Romain Huet, Rory Carmichael, Rowan Zellers, Roy Chen, Ruby Chen, Ruslan Nigmatullin, Ryan Cheu, Saachi Jain, Sam Altman, Sam Schoenholz, Sam Toizer, Samuel Miserendino, Sandhini Agarwal, Sara Culver, Scott Ethersmith, Scott Gray, Sean Grove, Sean Metzger, Shamez Hermani, Shantanu Jain, Shengjia Zhao, Sherwin Wu, Shino Jomoto, Shirong Wu, Shuaiqi, Xia, Sonia Phene, Spencer Papay, Srinivas Narayanan, Steve Coffey, Steve Lee, Stewart Hall, Suchir Balaji, Tal Broda, Tal Stramer, Tao Xu, Tarun Gogineni, Taya Christianson, Ted Sanders, Tejal Patwardhan, Thomas Cunningham, Thomas Degry, Thomas Dimson, Thomas Raoux, Thomas Shadwell, Tianhao Zheng, Todd Underwood, Todor Markov, Toki Sherbakov, Tom Rubin, Tom Stasi, Tomer Kaftan, Tristan Heywood, Troy Peterson, Tyce Walters, Tyna Eloundou, Valerie Qi, Veit Moeller, Vinnie Monaco, Vishal Kuo, Vlad Fomenko, Wayne Chang, Weiye Zheng, Wenda Zhou, Wesam Manassra, Will Sheu, Wojciech Zaremba, Yash Patil, Yilei Qian, Yongjik Kim, Youlong Cheng, Yu Zhang, Yuchen He, Yuchen Zhang, Yujia Jin, Yunxing Dai, and Yury Malkov. 2024. GPT-4o System Card. *arXiv:2410.21276 [cs.CL]* <https://arxiv.org/abs/2410.21276>
- Jan Peters and Stefan Schaal. 2007. Reinforcement learning by reward-weighted regression for operational space control. *Proceedings of the 24th Annual International Conference on Machine Learning (ICML 2007)*, 745-750 (2007) (06 2007). <https://doi.org/10.1145/1273496.1273590>
- Minh VT Pham, Huy N Phan, Hoang N Phan, Cuong Le Chi, Tien N Nguyen, and Nghi DQ Bui. 2025. SWE-Synth: Synthesizing Verifiable Bug-Fix Data to Enable Large Language Models in Resolving Real-World Bugs. *arXiv preprint arXiv:2504.14757* (2025).
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *arXiv:2305.18290 [cs.LG]* <https://arxiv.org/abs/2305.18290>
- Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. 2020. Test-time training with self-supervision for generalization under distribution shifts. In *International conference on machine learning*. PMLR, 9229–9248.
- Stanford University. 2023. CodeAlpaca: An Instruction-following LLaMA Model for Programming Tasks. <https://github.com/sahil280114/codealpaca>
- Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. 2020. TRL: Transformer Reinforcement Learning. <https://github.com/huggingface/trl>.
- Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. 2020. Tent: Fully test-time adaptation by entropy minimization. *arXiv preprint arXiv:2006.10726* (2020).

- Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. Magicoder: Empowering code generation with oss-instruct. *arXiv preprint arXiv:2312.02120* (2023).
- Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore, Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 959–971. <https://doi.org/10.1145/3540250.3549101>
- John Yang, Kilian Leret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. SWE-smith: Scaling Data for Software Engineering Agents. *arXiv preprint arXiv:2504.21798* (2025).
- Zibin Zheng, Kaiwen Ning, Qingyuan Zhong, Jiachi Chen, Wenqing Chen, Lianghong Guo, Weicheng Wang, and Yanlin Wang. 2024. Towards an Understanding of Large Language Models in Software Engineering Tasks. *arXiv:2308.11396 [cs.SE]* <https://arxiv.org/abs/2308.11396>
- Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, Binyuan Hui, Niklas Muennighoff, David Lo, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. *arXiv:2406.15877 [cs.SE]* <https://arxiv.org/abs/2406.15877>